

10. Dimensionality reduction and auto-encoders

9.12.2025

ETH zürich

D MATH

In today's class...

10.1 Motivation

10.2 Principal Component Analysis (PCA)

- Example: MNIST dataset
- Example: yield curves

10.3 Kernel PCA

10.4 Auto-encoders

- Sparse auto-encoders
- Denoising auto-encoders
- Contractive auto-encoders
- Example: Categorical feature embedding

10.1 Motivation

Dimensionality reduction is the problem of representing high-dimensional data in a low-dimensional space (called the ***latent space***) by preserving as much information as possible.

The latent representations can be used for:

- data compression
- clustering
- data visualization

But they can also be useful for solving downstream tasks (such as classification):

- it is computationally more efficient to work in low-dimensions
- it alleviates the curse of dimensionality (high-dimensional data is very sparse)
- latent representations correspond to meaningful (or structural) features of the data

10.2 Principal Component Analysis (PCA)

Main idea: project the data onto a low-dimensional subspace while preserving the maximum amount of variance.

- Consider a data set $\{(x_{i,1}, \dots, x_{i,d}), 1 \leq i \leq m\}$ of m iid realizations of a random vector $X = (X_1, \dots, X_d)$ with design matrix:

$$A = \begin{pmatrix} x_{1,1} & \dots & x_{1,d} \\ x_{2,1} & \dots & x_{2,d} \\ \vdots & \ddots & \vdots \\ x_{m,1} & \dots & x_{m,d} \end{pmatrix}$$

- We center the columns of A by setting $x_{i,j} \mapsto x_{i,j} - \bar{x}_j$, where $\bar{x}_j = \frac{1}{m} \sum_{i=1}^m x_{i,j}$.
- Depending on the application, we may also standardize the columns of A by setting $x_{i,j} \mapsto \frac{x_{i,j} - \bar{x}_j}{\hat{\sigma}_j}$, where $\hat{\sigma}_j = \sqrt{\frac{1}{m} \sum_{i=1}^m (x_{i,j} - \bar{x}_j)^2}$.
- We look for an orthonormal basis $v_1, \dots, v_d$ of $\mathbb{R}^d$ such that:
 - v_1 is the direction of highest variance in the data (i.e. v_1 maximizes $\text{Var}\langle w, X \rangle$ among all vectors $w \in \mathbb{R}^d$ with $\|w\|_2 = 1$),
 - v_2 is the direction of second-highest variance orthogonal to v_1 and so on...

10.2 Principal Component Analysis (PCA)

- We must solve recursively the following optimization problems:

$$v_1 = \arg \max_{\|w\|_2=1} \|Aw\|_2^2 \quad \text{and} \quad v_k = \arg \max_{\substack{\|w\|_2=1 \\ v_j^T w=0, \text{ for all } j < k}} \|Aw\|_2^2, \quad \text{for all } 1 \leq k \leq d.$$

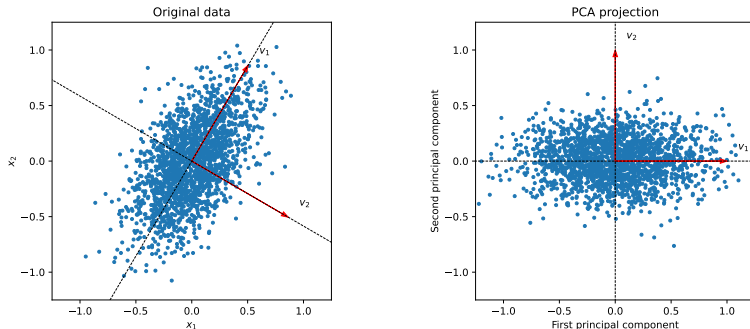
- We start by noticing that:

$$v_1 = \arg \max_{\|w\|_2=1} \|Aw\|_2^2 = \arg \max_{\|w\|_2=1} w^T A^T A w = \arg \max_{\|w\|_2=1} w^T V \operatorname{diag}(\sigma_1^2, \dots, \sigma_d^2) V^T w,$$

where we eigendecomposed $A^T A$ (the sample covariance matrix, up to a multiplicative factor), with V an orthogonal matrix whose columns are the eigenvectors of $A^T A$ and with $\sigma_1^2 \geq \dots \geq \sigma_d^2$ the eigenvalues of $A^T A$ in decreasing order.

- Since $\|w^T V\|_2 = \|w\|_2 = 1$ (by orthogonality of V), the optimal w must satisfy $w^T V = (1, 0, \dots, 0)$, hence it corresponds to v_1 , the first eigenvector of $A^T A$.
- Using a similar reasoning for $k > 1$ and imposing the orthogonality constraints $\langle v_j, v_k \rangle = 0$ for all $j < k$, we obtain that the solutions $v_1, \dots, v_d$ must be the columns of V , i.e. the eigenvectors of $A^T A$ corresponding to the (ordered) eigenvalues $\sigma_1^2 \geq \sigma_2^2 \geq \dots \geq \sigma_d^2$.

10.2 Principal Component Analysis (PCA)



- The eigenvectors $(v_1, \dots, v_d)$ are called the *principal components* of the data.
- We can represent each data point $x_i = (x_{i,1}, \dots, x_{i,d})$ in the basis $(v_1, \dots, v_d)$:

$$x_i \mapsto (x_i^T v_1, \dots, x_i^T v_d).$$

- The coordinates $x_i^T v_1, \dots, x_i^T v_d$ are called the *principal component scores* of the data point x_i .
- **Attention!** Some authors call $(x_i^T v_1, \dots, x_i^T v_d)$ the principal components and call $(v_1, \dots, v_d)$ the loadings.

10.2 Principal Component Analysis (PCA)

PCA is connected with the Singular Value Decomposition (SVD) of the design matrix A :

- The square roots of the eigenvalues of $A^T A$, $\sigma_1, \dots, \sigma_d$, are called the **singular values** of A .
- In Lecture 2 we saw that if A has $r \leq d$ strictly positive singular values $\sigma_1 \geq \sigma_2 \geq \dots \sigma_r > 0$, then A admits the following Singular Value Decomposition (SVD):

$$A = U \Sigma V^T = \sum_{i=1}^r \sigma_i u_i v_i^T.$$

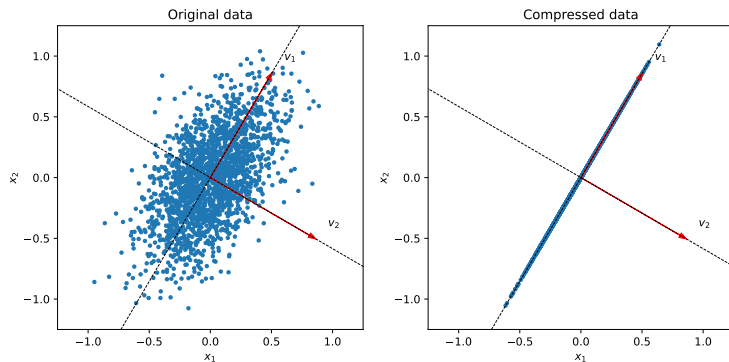
where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{d \times d}$ are orthogonal matrices and $\Sigma \in \mathbb{R}^{m \times d}$ is a diagonal matrix with $\sigma_1, \dots, \sigma_r$ on the main diagonal.

- The eigendecomposition of $A^T A$ can be recovered from the SVD of A :

$$A^T A = (U \Sigma V^T)^T (U \Sigma V^T) = V \Sigma^T U^T U \Sigma V^T = V \Sigma^T \Sigma V^T$$

- Therefore the columns of V are exactly the eigenvectors $v_1, \dots, v_d$ of $A^T A$.
- It is in general more efficient to perform PCA starting from the SVD of A , because fast algorithms for SVD exist that do not require computing $A^T A$ [Golub and Van Loan, 2013].

10.2 Principal Component Analysis (PCA)



- We can obtain a low-dimensional representation of the data by keeping the first $k \leq d$ principal components:

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T = U_k \Sigma_k V_k^T, \quad (\text{truncated SVD})$$

where $\Sigma_k = \text{diag}(\sigma_1, \dots, \sigma_k) \in \mathbb{R}^k$, and $U_k \in \mathbb{R}^{m \times k}$ and $V_k \in \mathbb{R}^{d \times k}$ are obtained from U and V by keeping the first k columns.

10.2 Principal Component Analysis (PCA)

- A_k is the unique minimizer of the Frobenius distance from the matrix A among all matrices of rank at most k .

Theorem 1 ([Eckart and Young, 1936])

Let $A_k := \sum_{i=1}^k \sigma_i u_i v_i^T = U_k \Sigma_k V_k^T$ be the truncated SVD of the design matrix $A \in \mathbb{R}^{m \times d}$, then

$$A_k = \arg \min_{\substack{B \in \mathbb{R}^{m \times d} \\ \text{rank}(B) \leq k}} \|A - B\|_F,$$

where $\|A\|_F^2 = \sum_{i=1}^d \sum_{j=1}^m a_{i,j}^2 = \text{Tr}(A^T A)$ is the Frobenius norm of A .

Remark:

- For any matrix $A \in \mathbb{R}^{m \times d}$ with SVD $A = U \Sigma V^T$ we have:

$$\|A\|_F = \|U \Sigma V^T\|_F = \text{Tr}(V \Sigma^T U^T U \Sigma V^T) = \text{Tr}(V \Sigma^T \Sigma V^T) = \text{Tr}(V^T V \Sigma^T \Sigma) = \text{Tr}(\Sigma^T \Sigma) = \sum_{i=1}^d \sigma_i^2(A),$$

where $\sigma_i(A)$ is the i -th singular value of matrix A .

10.2 Principal Component Analysis (PCA)

Proof.

We start by noticing that the truncated SVD satisfies $\|A - A_k\|_F^2 = \left\| \sum_{i=k+1}^d \sigma_i u_i v_i^T \right\|_F^2 = \sum_{i=k+1}^d \sigma_i^2$.

Let $B \in \mathbb{R}^{m \times d}$ be any matrix of rank at most k . Weyl's inequality for singular values states that

$$\sigma_{i+j-1}(X + Y) \leq \sigma_i(X) + \sigma_j(Y),$$

which for $X = A - B$, $Y = B$ and $j = k + 1$ reduces to:

$$\sigma_{k+i} = \sigma_{k+i}(A) \leq \sigma_i(A - B) + \sigma_{k+1}(B) = \sigma_i(A - B),$$

where we used that $\sigma_{k+1}(B) = 0$, because $\text{rank}(B) \leq k$.

Therefore:

$$\|A - B\|_F^2 = \sum_{i=1}^d \sigma_i^2(A - B) \geq \sum_{i=1}^{d-k} \sigma_i^2(A - B) \geq \sum_{i=k+1}^d \sigma_i^2 = \|A - A_k\|_F^2$$



Consequences of the theorem:

- A_k is the only matrix of rank at most k that preserves a fraction $\sum_{i=1}^k \sigma_i^2 / \sum_{i=1}^d \sigma_i^2$ of the total variance in the original data (fraction of *explained variance*).
- Any other matrix of rank at most k is suboptimal, in the sense that it preserves less variance.

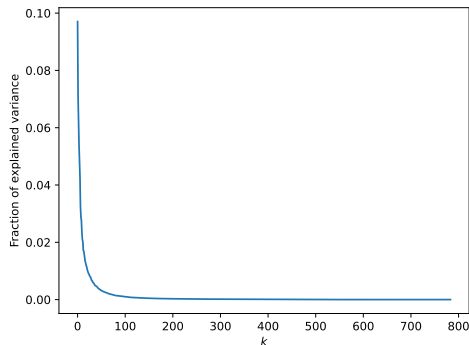
10.2 Example: MNIST dataset

- For some kinds of data (such as images) high compression rates can be attained (images concentrate on low-dimensional manifolds in pixel space).
- We present a small application of PCA to the MNIST dataset, which consists of 70,000 black and white pictures of handwritten digits:



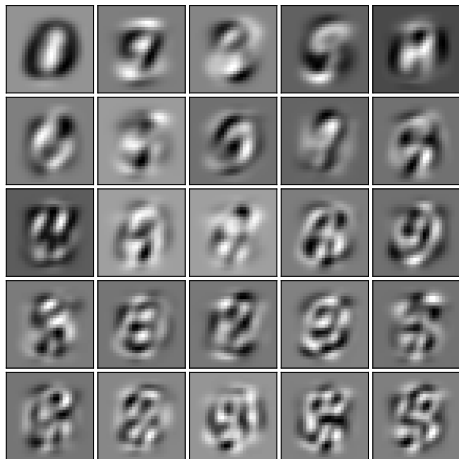
10.2 Example: MNIST dataset

- We perform PCA on the “flattened” images (28×28 image $\mapsto$ 784-dimensional vector).



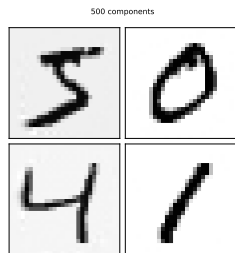
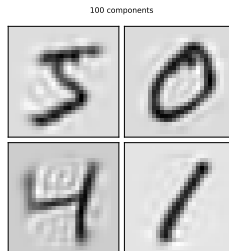
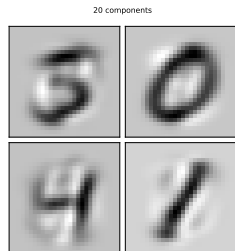
- Choose optimal number of principal components k^* by plotting the ***fraction of variance explained*** by each component $k \mapsto \sigma_k^2 / \sum_{i=1}^d \sigma_i^2$ and:
 - Choose k^* such that $\sum_{i=k^*+1}^d \sigma_i^2 / \sum_{i=1}^d \sigma_i^2 < \text{threshold}$ (e.g. 1%),
 - **Elbow method**: choose k^* such that the curve flattens abruptly for $k > k^*$.
- For the MNIST dataset the first 22 principal components already explain 99% of the variance.

10.2 Example: MNIST dataset



- We plot the first 25 principal components (decreasing order: top to bottom, left to right)
- The first few components capture features that are recognizable as digits (or parts of digits), while higher components capture finer details (and eventually noise).

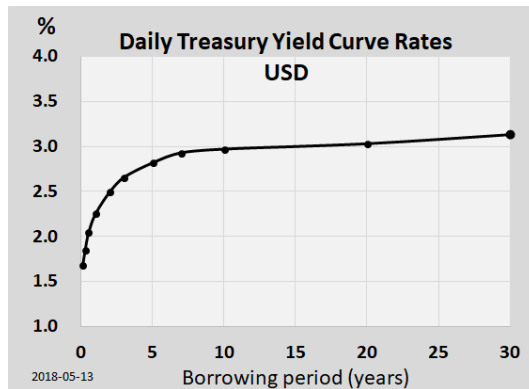
10.2 Example: MNIST dataset



- Keeping a number $k \in \{20, 100, 500, 784\}$ of principal components allows to compress the data and reconstruct it at different levels of detail.

10.2 Example: yield curves

- A **yield curve** (or **term structure** of interest rates) is a curve depicting the **yields**¹ of a class of instruments at different maturities.
- For example, below is the yield curve $(y(t, T_1), y(t, T_2), \dots, y(t, T_d))$ at time $t = 2018.05.13$ for US Treasury bonds for a collection of maturities $T_1, T_2, \dots, T_d$.



¹The yield is a measure of the total return from buying an instrument (typically a bond) and holding it up to its maturity. The yield is expressed in percentage points, like an interest rate.

10.2 Example: yield curves

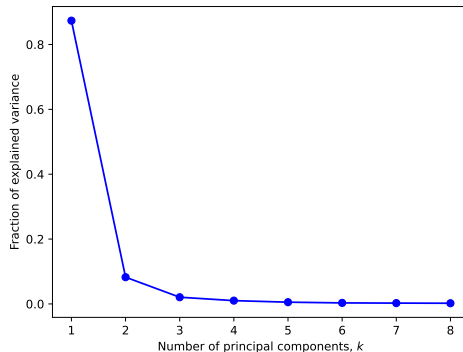
- Each day the yield curve changes, because the market prices of the underlying instruments change.
- Institutional bond investors try to invest in such a way that the value of their portfolios is robust to small changes in the yield curve (*portfolio immunization*), this requires understanding how the yield curve moves in time.
- Instead of studying the changes of individual yields $y(t, T)$ for each maturity T separately, we can analyze the evolution of the entire yield curve as a high-dimensional object by computing the PCA of its joint changes:

$$(y(t_{i+1}, T_1) - y(t_i, T_1), \dots, y(t_{i+1}, T_d) - y(t_i, T_d)), \quad 1 \leq i \leq m,$$

from past observation dates $(t_1, t_2, \dots, t_m)$.

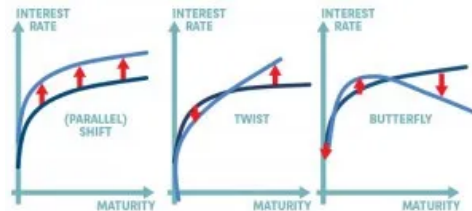
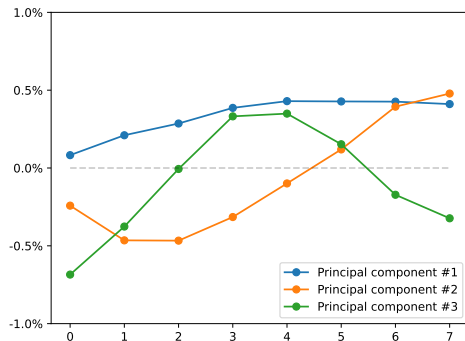
10.2 Example: yield curves

- We perform PCA on a data set of 2,631 daily yield curve changes from January 2010 to July 2020 for 8 different maturities (1, 2, 3, 5, 7, 10, 20 and 30 years).



- The first principal component alone explains 87% of the variance.
- The first three principal components cumulatively explain more than 97% of the variance.

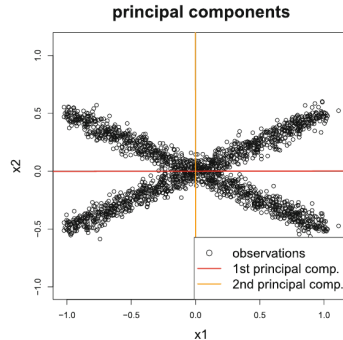
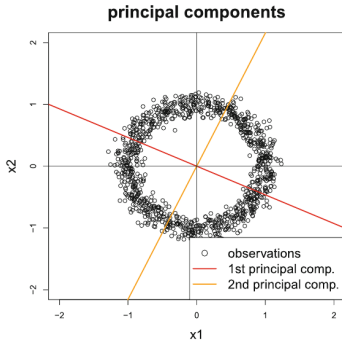
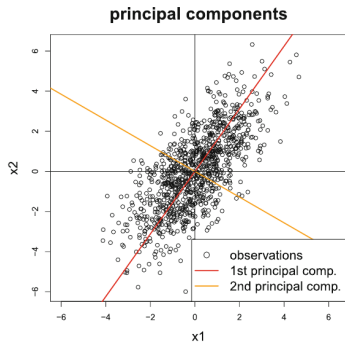
10.2 Example: yield curves



- The first three principal components (left) are well-known to practitioners (right).
- Principal component #1: the *shift*, is a roughly parallel shift of the curve.
- Principal component #2: the *twist*, is a rotation of the curve (downwards for low maturities, upwards for high maturities).
- Principal component #3: the *butterfly*, is a “bowing” of the curve (downwards for low and high maturities, upwards for medium maturities)

10.3 Kernel PCA

PCA is ultimately a linear transformation of the input data (it is a projection on the first k principal components) and may fail to capture data variability in non-linear data sets²:



²Image taken from [Wüthrich and Merz, 2023]

10.3 Kernel PCA

- [Schölkopf et al., 1997] proposed to perform PCA using kernel methods.
- Let H be an RKHS with feature map $\Phi : \mathcal{X} \rightarrow H$ and reproducing kernel k and assume that the features $\Phi(x_1), \dots, \Phi(x_m)$ are centered (i.e. $\sum_{i=1}^m \Phi(x_i) = 0$).
- We consider the covariance of the transformed features, which is the linear operator C on H given by $C w = \sum_{i=1}^m \langle w, \Phi(x_i) \rangle_H \Phi(x_i)$.
- We can perform non-linear PCA by finding the eigendecomposition of C , i.e. we look for eigenvectors $v \in H \setminus \{0\}$ with eigenvalues $\lambda \geq 0$, such that $Cv = \lambda v$.
- Since all solutions v must lie in the span of $\Phi(x_1), \dots, \Phi(x_m)$, we can equivalently solve

$$\langle \Phi(x_k), Cv \rangle_H = \lambda \langle \Phi(x_k), v \rangle_H, \quad \text{for all } k = 1, \dots, m,$$

where we take $v = \sum_{i=1}^m \alpha_i \Phi(x_i)$.

- Defining $K_{i,j} := \langle \Phi(x_i), \Phi(x_j) \rangle_H$ and $\alpha = (\alpha_1, \dots, \alpha_m)$, the problem reduces to

$$K\alpha = \lambda\alpha,$$

which is equivalent to finding the eigenvectors $(\alpha^{(1)}, \dots, \alpha^{(m)})$ of K .

10.3 Kernel PCA

- We compute each eigenvector $v_k = \sum_{i=1}^m \alpha_i^{(k)} \Phi(x_i)$ with eigenvalue λ_k and we normalize it in H by imposing:

$$1 = \langle v_k, v_k \rangle_H = \sum_{i,j=1}^m \alpha_i^{(k)} \alpha_j^{(k)} \langle \Phi(x_i), \Phi(x_j) \rangle_H = \langle \alpha^{(k)}, K \alpha^{(k)} \rangle = \lambda_k \langle \alpha^{(k)}, \alpha^{(k)} \rangle$$

- Finally, we can compute the kernel principal component scores of a data point x as:

$$\langle v_k, \Phi(x) \rangle_H = \sum_{i=1}^m \alpha_i^{(k)} \langle \Phi(x_i), \Phi(x) \rangle_H = \sum_{i=1}^m \alpha_i^{(k)} k(x_i, x), \quad \text{for } k \leq m.$$

Remarks:

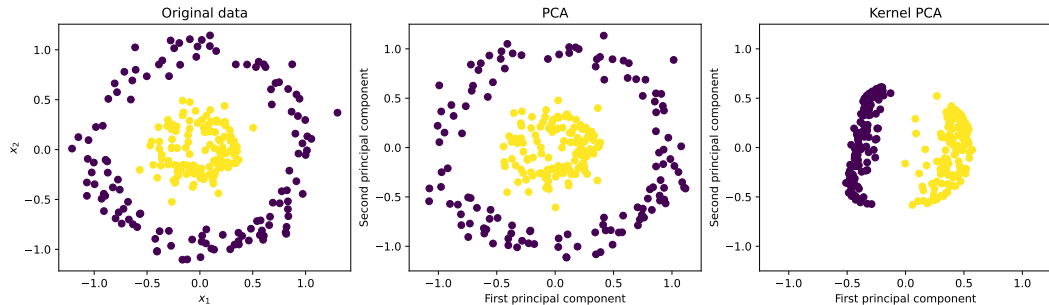
- As typical for kernel methods, we never have to compute the features $\Phi(x_1), \dots, \Phi(x_m)$ explicitly. We only need to be able to evaluate the kernel function $k(x, x') = \langle \Phi(x), \Phi(x') \rangle_H$.
- If $\Phi(x_1), \dots, \Phi(x_m)$ are not centered we repeat the above steps for $\tilde{\Phi}(x_i) = \Phi(x_i) - \frac{1}{m} \sum_{i=1}^m \Phi(x_i)$ and obtain that the matrix to eigendecompose is

$$\tilde{K} = K - U_m K - K U_m + U_m K U_m,$$

where $(U_m)_{i,j} = 1/m$.

10.3 Kernel PCA

- We perform kernel PCA on a non-linear toy data set for binary classification.
- Data points are labelled (either yellow or blue) and have a strongly non-linear distribution.



- Classical PCA fails to extract a meaningful representation of the data.
- Using a Gaussian kernel $k(x, x') = \exp(-\|x - x'\|^2/\gamma^2)$ with $\gamma = 2$, the first two principal components of kernel PCA make the projected data linearly separable.

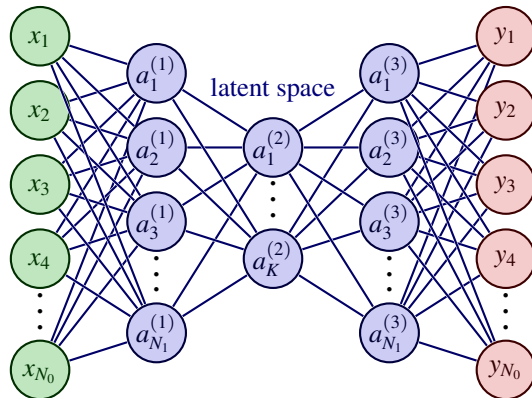
10.4 Auto-encoders

- Many authors proposed using neural networks to extract non-linear features from the data in an unsupervised way [Le Cun and Fogelman-Soulié, 1987], [Bourlard and Kamp, 1988], [Hinton and Zemel, 1993].
- [Kramer, 1991] introduced *auto-encoders* (also known as *bottleneck neural networks*) as a non-linear version of PCA.
- [Hinton and Salakhutdinov, 2006] were the first to train a deep auto-encoder which had better performance than PCA on the MNIST dataset.

Using neural networks has many advantages:

- the latent representation is learned automatically from data,
- the compression is strongly data-specific (i.e. optimal for the training set).

10.4 Auto-encoders

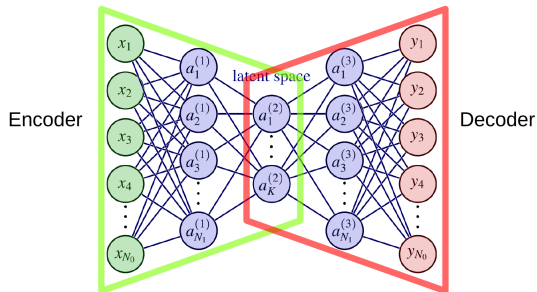


- The network has a symmetric “bottleneck” architecture, for example:

$$(N_0, N_1, K, N_1, N_0) = (100, 25, 5, 25, 100)$$

- The middle layer is the **latent space** and it has dimension $K \ll N_0$.

10.4 Auto-encoders



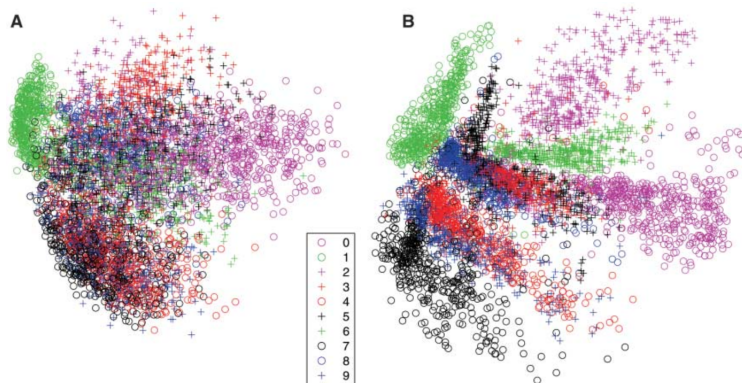
- The network is trained on a data set $(x_i \in \mathbb{R}^{N_0}, 1 \leq i \leq m)$ by minimizing a *dissimilarity function* $\ell : \mathbb{R}^{N_0} \times \mathbb{R}^{N_0} \rightarrow \mathbb{R}_+$ with $\ell(x, x') = 0$ if and only if $x = x'$:

$$\arg \min_{(\theta, \bar{\theta})} \frac{1}{m} \sum_{i=1}^m \ell(\mathbf{G}_{\bar{\theta}}(\mathbf{F}_{\theta}(x_i)), x_i),$$

where $\mathbf{F}_{\theta} : \mathbb{R}^{N_0} \rightarrow \mathbb{R}^K$ is the *encoder* and $\mathbf{G}_{\bar{\theta}} : \mathbb{R}^K \rightarrow \mathbb{R}^{N_0}$ is the *decoder*.

- By learning how to optimally encode the input, the network is forced to extract meaningful (i.e. structural) data features.

10.4 Auto-encoders



We can compare PCA and a deep auto-encoder trained on the MNIST dataset³:

- (A) Data projection on first and second principal components from PCA,
- (B) Data projection on the two-dimensional latent space of an auto-encoder with architecture (784, 1000, 500, 250, 2, 250, 500, 1000, 784).

³Image taken from [[Hinton and Salakhutdinov, 2006](#)]

10.4 Auto-encoders

- We want auto-encoders to have enough capacity to extract meaningful features, but not so much capacity that they simply learn the identity or memorize the dataset.
- Intuitively, the following two limit cases must be avoided:
 - One-dimensional latent space ($K = 1$) and very complex decoder/encoder
 $\implies$ the decoder can memorize the data by learning the map $i \mapsto x_i$ for all data points $1 \leq i \leq m$.
 - Large latent space ($K = N_0$)
 $\implies$ encoder/decoder can just learn the identity function.
- Good feature extraction requires a low latent dimension and regularizing the encoder and decoder.
- Different regularizations correspond to different architectures:
 - sparse auto-encoders (SAEs)
 - denoising auto-encoders (DAEs)
 - contractive auto-encoders (CAEs)

10.4 Auto-encoders: sparse auto-encoders

- Sparse auto-encoders include a penalty term for the latent activations:

$$\ell(x, G_{\tilde{\theta}}(F_{\theta}(x))) + \mathcal{R}(F_{\theta}(x)).$$

- This is a non-standard form of regularization, because it does not act on model's parameters (as, for example, the L^2 regularizer $\mathcal{R}(\theta) = \|\theta\|_2^2$).
- The penalty $\mathcal{R}$ encourages *sparse latent representations*, i.e. $F_{\theta}(x)$ should have most coordinates equal to zero.
- Each coordinate of $F_{\theta}(\cdot)$ specializes to encode selected data patterns and is prevented from learning the identity function.

Typical choices of sparsity:

- Laplace prior or L^1 regularization: $\mathcal{R}(F_{\theta}(x)) = \lambda \|F_{\theta}(x)\|_1$
- d -sparse regularization: apply a sparse activation function ρ_d to the latent layer:

$$\rho_d(F_1(x), \dots, F_K(x)) = (b_1 F_1(x), \dots, b_K F_K(x)),$$

where $b_j = 1$ if and only if $|F_j(x)|$ is among the d highest values of $\{|F_1(x)|, \dots, |F_K(x)|\}$, otherwise $b_j = 0$.

10.4 Auto-encoders: denoising auto-encoders

- Instead of minimizing $\ell(x, G_{\tilde{\theta}}(F_{\theta}(x)))$, denoising auto-encoders are trained by minimizing

$$\ell(x, G_{\tilde{\theta}}(F_{\theta}(\tilde{x}))), \quad (1)$$

where $\tilde{x}$ is a copy of x that has been corrupted through some noise distribution $C(\cdot|x)$.

- Training procedure:
 1. Sample a training sample x from the training data set.
 2. Sample a corrupted copy $\tilde{x} \sim C(\cdot|x)$ of x .
 3. Use $(\tilde{x}, x)$ as training sample for the auto-encoder as in Equation (1).
- Typical noise distributions are:
 - Additive Gaussian noise: $C(\cdot|x) = x + \varepsilon$, where $\varepsilon \sim \mathcal{N}(0, \sigma_{\varepsilon}^2)$, for some small $\sigma_{\varepsilon} > 0$.
 - Salt-and-pepper noise: a fraction of the pixels is randomly set to white or black.
- The auto-encoder is trained not just to reconstruct the input well, but also to revert noise corruption and can be used to denoise documents and images.



10.4 Auto-encoders: contractive auto-encoders

- Contractive auto-encoders have a loss of the form:

$$\ell(x, G_{\tilde{\theta}}(F_{\theta}(x))) + \lambda \|\nabla_x F_{\theta}(x)\|_F^2.$$

- This penalty encourages the model to learn latent representation that locally do not change much for close inputs x .
- Similarly to denoising auto-encoders, contractive auto-encoders have latent representations that are robust to infinitesimal perturbations of the input (this equivalence is exact in the case of Gaussian noise [Alain and Bengio, 2014]).
- When $F_{\theta}(\cdot)$ has sigmoidal activation function, the penalty term forces the sigmoidal units to saturate to 0 or 1, which results in a sparse latent representation.

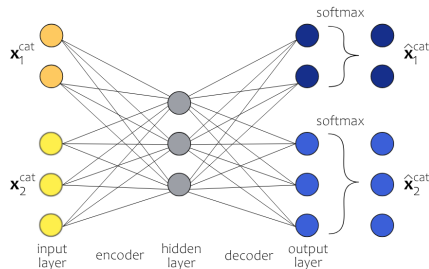
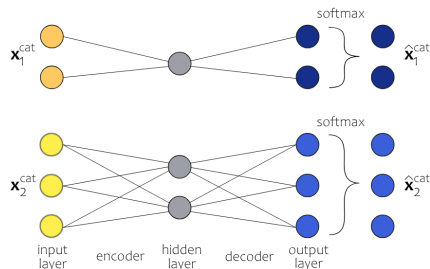
10.4 Example: categorical feature embedding

- It is common for insurance data sets to have many categorical features (as in Project 4).
- A typical solution is to one-hot-encode a categorical feature with d levels $\{v_1, \dots, v_d\}$ into $\mathbb{R}^d$:

$$V \ni v_i \mapsto e_i, \quad 1 \leq i \leq d$$

where e_i is the i -th standard basis vector in $\mathbb{R}^d$.

- This may lead to an unacceptably high input space dimension. Instead, we can compress the one-hot encoded features — either separately (left) or jointly (right) — using auto-encoders⁴:



- The latent representations of the features (instead of the features themselves) are then used as inputs to the insurance model.

⁴Image taken from [Delong and Kozak, 2023]

References I

-  Alain, G. and Bengio, Y. (2014).
What regularized auto-encoders learn from the data-generating distribution.
The Journal of Machine Learning Research, 15(1):3563–3593.
-  Bourlard, H. and Kamp, Y. (1988).
Auto-association by multilayer perceptrons and singular value decomposition.
Biological cybernetics, 59(4-5):291–294.
-  Delong, Ł. and Kozak, A. (2023).
The use of autoencoders for training neural networks with mixed categorical and numerical features.
ASTIN Bulletin: The Journal of the IAA, 53(2):213–232.
-  Eckart, C. and Young, G. (1936).
The approximation of one matrix by another of lower rank.
Psychometrika, 1(3):211–218.

References II

-  Golub, G. H. and Van Loan, C. F. (2013).
Matrix computations.
JHU press.
-  Hinton, G. E. and Salakhutdinov, R. R. (2006).
Reducing the dimensionality of data with neural networks.
science, 313(5786):504–507.
-  Hinton, G. E. and Zemel, R. (1993).
Autoencoders, minimum description length and helmholtz free energy.
Advances in neural information processing systems, 6.
-  Kramer, M. A. (1991).
Nonlinear principal component analysis using autoassociative neural networks.
AIChE journal, 37(2):233–243.
-  Le Cun, Y. and Fogelman-Soulié, F. (1987).
Modèles connexionnistes de l'apprentissage.
Intellectica, 2(1):114–143.

References III

-  Schölkopf, B., Smola, A., and Müller, K.-R. (1997).
Kernel principal component analysis.
In International conference on artificial neural networks, pages 583–588. Springer.
-  Wüthrich, M. V. and Merz, M. (2023).
Statistical foundations of actuarial learning and its applications.
Springer Nature.